

E-COMMERCE MARKETPLACE

Implementation Plan

Full-Stack Development Roadmap · Node.js · Express · MongoDB · EJS · Tailwind CSS

Version 1.0 · June 22, 2026 · Status: Draft · Classification: Confidential

Version v1.0	Phases 6	Est. Duration 16 Weeks	Collections 10	API Routes 40+	Team Size 3–5 Devs
--------------	----------	---------------------------	----------------	-------------------	-----------------------

Table of Contents

1.	Executive Summary	3
2.	Project Overview & Architecture	3
3.	Phase 1 — Foundation & Project Setup	4
4.	Phase 2 — Authentication & User Management	5
5.	Phase 3 — Core Product & Catalogue	6
6.	Phase 4 — Commerce Engine (Cart, Checkout, Orders)	7
7.	Phase 5 — Advanced Features (AI, Reviews, Support)	9
8.	Phase 6 — Admin Portal, Polish & Deployment	11
9.	API Route Summary	12
10.	Milestone & Delivery Schedule	13
11.	Risk Register	14
12.	Tech Stack & Dependencies	15
13.	Testing Strategy	15
14.	Definition of Done	16

1. Executive Summary

This Implementation Plan is the master development roadmap for the E-Commerce Marketplace Platform — a full-stack, multi-role marketplace built on Node.js, Express.js, MongoDB, EJS, and Tailwind CSS. It translates the Product Requirements Document (PRD), Technical Requirements Document (TRD), Backend Schema Reference, and UI/UX Brief into a structured, phased execution plan across six phases spanning approximately 16 weeks.

The platform serves three user roles — Customer, Seller, and Admin — and delivers AI-powered features including a chatbot assistant, voice search, and image-based product discovery. The backend schema defines 10 MongoDB collections managed via Mongoose ODM, with a RESTful API layer, JWT-based authentication, and Cloudinary media integration. The UI adopts a Minimalist + Glassmorphism aesthetic with full dark-mode support.

Attribute	Detail
Stack	Node.js · Express.js · MongoDB (Mongoose) · EJS · Tailwind CSS · JWT
Roles	Customer · Seller · Admin (Hidden Panel)
Collections	User · Product · Cart · Wishlist · Order · Review · Coupon · Notification · SupportTicket · ChatHistory
AI Features	AI Chatbot (Claude/OpenAI) · Voice Search (Web Speech API) · Image Search (Vision API)
Phases	6 phases — Foundation → Auth → Products → Commerce → Advanced → Admin/Deploy
Total Duration	~16 weeks (adjustable based on team size)
Architecture	Monolithic MVC — Express.js with EJS server-side rendering; REST API (/api/v1)

2. Project Overview & Architecture

2.1 High-Level Architecture

The platform is a monolithic MVC application. Express.js handles routing and business logic; EJS renders server-side HTML pages enriched by Tailwind CSS. MongoDB (via Mongoose ODM) is the sole data store. REST API endpoints (/api/v1) serve JSON for dynamic client-side interactions. JWT tokens are stored as HttpOnly cookies for session management.

2.2 Directory Structure

Path	Purpose
/	Project root — package.json, .env, tailwind.config.js, server.js
/config	db.js (Mongoose connection), cloudinary.js, env validation
/models	10 Mongoose model files + index.js barrel export
/controllers	auth, product, cart, order, seller, admin, review, coupon, notification, support, chat
/routes	Mirrors controllers; all prefixed /api/v1 or ADMIN_PATH
/middleware	auth.js (JWT verify), roleGuard.js, errorHandler.js, upload.js (Multer)
/services	CartService, OrderService, AuthService, CouponService, NotificationService
/views	EJS templates — layout, partials, customer, seller, admin subdirs
/public	dist/tailwind.css, images, client-side JS bundles
/utils	email.js (Nodemailer), helpers.js, validators.js, logger.js

3. Phase 1 — Foundation & Project Setup

Phase 1	Foundation & Project Setup	Weeks 1–2 · ~10 days
---------	----------------------------	----------------------

Establish the complete development scaffolding before any feature work begins. Every subsequent phase depends on the environment, database connection, and project conventions defined here.

3.1 Tasks

#	Task	Owner	Est. Days	Dependency	Priority
1.1	Initialise Node.js project — npm init, install core dependencies (express, mongoose, ejs, dotenv, bcryptjs, jsonwebtoken, multer, cloudinary, nodemailer, morgan)	Backend	1	—	High
1.2	Configure Tailwind CSS build pipeline — tailwind.config.js, PostCSS, watch script, dark mode class strategy	Frontend	1	—	High
1.3	Set up MongoDB Atlas cluster, create database, configure Mongoose connection in /config/db.js with retry logic	Backend	0.5	1.1	High
1.4	Define .env schema and validation (dotenv + joi/invalid) — all environment variables per TRD §3.1	Backend	0.5	1.1	High
1.5	Create Express app skeleton — server.js, app.js, global middleware (cors, cookie-parser, morgan, compression)	Backend	1	1.1	High
1.6	Set up EJS view engine with layout/partial partials structure (layout.ejs, navbar, footer)	Frontend	1	1.5	High
1.7	Implement all 10 Mongoose models per Backend Schema Reference — User, Product, Cart, Wishlist, Order, Review, Coupon, Notification, SupportTicket, ChatHistory	Backend	3	1.3	High
1.8	Create /models/index.js barrel export file	Backend	0.5	1.7	Medium
1.9	Configure Cloudinary SDK and Multer upload middleware for image handling	Backend	0.5	1.1	Medium
1.10	Set up global error handler middleware and standard API response envelope (success/error shape)	Backend	0.5	1.5	High
1.11	Configure Nodemailer transporter with email template helper	Backend	0.5	1.1	Medium

#	Task	Owner	Est. Days	Dependency	Priority
1.1 2	Set up Git repository, branching strategy (main/dev/feature), .gitignore, README	DevOps	0.5	—	High

3.2 Phase 1 Deliverables

- Runnable Express server with MongoDB connection
- All 10 Mongoose schemas defined, validated, and exported
- Tailwind CSS build pipeline with dark-mode support
- EJS layout with navbar and footer partials
- Git repository with branching strategy in place

4. Phase 2 — Authentication & User Management

Phase 2	Authentication & User Management	Weeks 3–4 · ~10 days
---------	----------------------------------	----------------------

Implement the complete authentication system covering customer signup/login, seller registration with approval workflow, JWT session management, password security, and account management. All role guard middleware is delivered in this phase.

4.1 Tasks

#	Task	Owner	Est. Days	Dependency	Priority
2.1	Implement JWT auth middleware — verify HttpOnly cookie, attach req.user, handle expired/invalid tokens	Backend	1	1.5	High
2.2	Implement roleGuard middleware factory — requireAuth, requireRole(customer seller admin)	Backend	0.5	2.1	High
2.3	POST /api/v1/auth/signup — validate input, hash password (bcrypt rounds=12), create User, set JWT cookie	Backend	1	2.1	High
2.4	POST /api/v1/auth/login — verify credentials, sign JWT { userId, role, email }, set HttpOnly cookie	Backend	0.5	2.3	High
2.5	POST /api/v1/auth/logout — clear auth_token cookie (Max-Age=0)	Backend	0.25	2.4	High
2.6	Email verification flow — generate verifyToken, send verification email, GET /api/v1/auth/verify/:token	Backend	1	1.11	Medium
2.7	Password reset flow — POST /forgot-password, POST /reset-password/:token with token expiry check	Backend	1	1.11	Medium
2.8	Seller registration form (EJS) — capture businessName, businessType, taxId, bankDetails; set sellerStatus=pending	Full Stack	1.5	2.3	High
2.9	Account deletion endpoint — BR-03 guard: query pending orders/complaints before User.deleteOne()	Backend	1	2.1	High
2.10	Customer profile page — view/update name, avatar (Cloudinary upload), phone, addresses CRUD	Full Stack	1.5	2.1	Medium
2.11	Sign Up / Log In modal UI — Glassmorphism style per UI/UX Brief §3.2; client-side validation	Frontend	1.5	2.3	High

#	Task	Owner	Est. Days	Dependency	Priority
2.1 2	Landing page (EJS) — hero section, CTA buttons to Sign Up, Log In, Browse as Guest per UI/UX Brief §3.1	Frontend	1	1.6	High
2.1 3	Password policy validation middleware — min 8 chars, uppercase, digit, special char	Backend	0.5	2.3	Medium

4.2 Phase 2 Deliverables

- Secure JWT-based login/signup with HttpOnly cookies
- Seller registration with pending approval state
- Email verification and password reset flows
- Role guard middleware protecting all downstream routes
- Landing page and auth modals with Glassmorphism UI

5. Phase 3 — Core Product & Catalogue

Phase 3	Core Product & Catalogue	Weeks 5–7 · ~13 days
---------	--------------------------	----------------------

Build the full product catalogue including listing, search (keyword, voice, image), filtering, product detail pages, comparison, and the seller product management portal. This phase delivers the storefront home and all discovery flows.

5.1 Tasks

#	Task	Owner	Est. Days	Dependency	Priority
3.1	Storefront home page (EJS) — featured products grid, category tiles, deals banner per UI/UX Brief §3.3; query isFeatured=true products	Full Stack	2	1.7	High
3.2	GET /api/v1/products — paginated product listing with filters: category, price range, rating, availability, sort	Backend	1.5	1.7	High
3.3	Full-text keyword search — GET /api/v1/products?q= using MongoDB \$text index on title/description/tags	Backend	1	3.2	High
3.4	Voice search integration — Web Speech API on client converts speech to text, fires keyword search query	Frontend	1	3.3	Medium
3.5	Image search integration — user uploads image; Vision API (Google Cloud Vision or similar) returns labels; mapped to product search query	Full Stack	2	3.3	Medium
3.6	Search results page (EJS) — filter sidebar, sort dropdown, product cards grid, pagination per UI/UX Brief §3.4	Frontend	1.5	3.2	High
3.7	Product detail page (EJS) — image gallery, specs table, stock badge, reviews section, Add to Cart / Wishlist CTAs per UI/UX Brief §3.5	Full Stack	2	3.2	High
3.8	Product comparison page — GET /api/v1/products/compare?ids= — side-by-side max 3 products per UI/UX Brief §3.6	Full Stack	1.5	3.7	Medium
3.9	Seller: POST /api/v1/seller/products — create product with Cloudinary image upload, slug auto-generation	Backend	1.5	1.9	High
3.10	Seller: PUT/DELETE /api/v1/seller/products/:id — update/unpublish own products; stock update triggers BR-04 auto-status	Backend	1	3.9	High

#	Task	Owner	Est. Days	Dependency	Priority
3.1 1	Seller dashboard — product list table, inventory manager, low-stock alerts, sales overview widgets per UI/UX Brief §4.2	Full Stack	2	3.9	High
3.1 2	Seller registration approval — admin approves/rejects sellerStatus; email notification sent via Nodemailer	Full Stack	1	2.8	High
3.1 3	Product pre-save middleware — slug generation, BR-04 stock/status sync (stock 0 → out_of_stock)	Backend	0.5	1.7	High

5.2 Phase 3 Deliverables

- Live storefront home with featured products and category browse
- Keyword, voice, and image search with filter/sort on results page
- Product detail page with full specs, gallery, and stock status
- Product comparison (up to 3 products side-by-side)
- Seller portal with product CRUD and inventory management

6. Phase 4 — Commerce Engine

Phase 4	Cart, Checkout, Orders & Payments	Weeks 8–11 · ~17 days
------------	-----------------------------------	-----------------------

Implement the full commerce pipeline from cart management through multi-step checkout, order placement, order lifecycle tracking, returns and refunds, wishlist, coupon engine, and notification system. This is the most complex phase, containing all transactional logic and business rules.

6.1 Cart Tasks

#	Task	Owner	Est. Days	Dependency	Priority
4.1	POST /api/v1/cart/add — CartService.addItem(): enforce BR-01 stock guard, snapshot priceAtAdd, upsert cart document	Backend	1.5	1.7	High
4.2	PUT /api/v1/cart/update — update item quantity; recompute subtotal/total virtuals	Backend	0.5	4.1	High
4.3	DELETE /api/v1/cart/remove/:productId — remove single item from cart	Backend	0.5	4.1	High
4.4	POST /api/v1/cart/coupon — validate coupon (BR-05 constraints), compute discount, store couponCode + discount	Backend	1	4.1	High
4.5	Cart page (EJS) — item list with qty controls, coupon field, order summary panel per UI/UX Brief §3.7	Frontend	1.5	4.1	High

6.2 Checkout & Order Tasks

#	Task	Owner	Est. Days	Dependency	Priority
4.6	Multi-step checkout page (EJS) — address selector, payment method, order summary, confirm step with animated transitions per UI/UX Brief §3.8	Frontend	2	4.5	High
4.7	POST /api/v1/orders — OrderService.checkout(): re-validate cart stock, snapshot address + item prices, decrement Product.stock (triggers BR-04), create Order, send confirmation email	Backend	2.5	4.1	High
4.8	Payment integration — card (Stripe/Razorpay), UPI, COD; paymentStatus update on webhook/callback	Backend	2	4.7	Medium
4.9	GET /api/v1/orders — customer order history with status badges per UI/UX Brief §3.9.1	Backend	0.5	4.7	High

#	Task	Owner	Est. Days	Dependency	Priority
4.1 0	GET /api/v1/orders/:id — order detail with StatusHistory timeline per UI/UX Brief §3.9.2	Backend	0.5	4.7	High
4.1 1	Order status lifecycle — Order pre-save middleware auto-appends StatusHistory on every status change	Backend	0.5	1.7	High
4.1 2	Seller order management — GET/PATCH /api/v1/seller/orders — view and update order status (processing/shipped/out_for_delivery/delivered)	Backend	1	4.7	High
4.1 3	Returns & Refunds — POST /api/v1/orders/:id/return — submit reason + Cloudinary photo upload; status → return_requested	Backend	1.5	4.7	High
4.1 4	Return approval/rejection — seller/admin PATCH route updates status; triggers notification + refund flow	Backend	1	4.13	High
4.1 5	Refund processing — on return_accepted, update paymentStatus=refunded, set refundAmount, trigger payment gateway refund	Backend	1	4.14	Medium

6.3 Wishlist, Coupon & Notification Tasks

#	Task	Owner	Est. Days	Dependency	Priority
4.1 6	POST/DELETE /api/v1/wishlist/toggle — add/remove using \$addToSet/\$pull; one wishlist per user	Backend	0.5	1.7	Medium
4.1 7	GET /api/v1/wishlist — retrieve wishlist with populated product refs; wishlist page EJS	Full Stack	1	4.16	Medium
4.1 8	Coupon management API — admin CRUD for coupons; Coupon.isUsable virtual + calculateDiscount() method	Backend	1	1.7	Medium
4.1 9	Notification service — create notifications on order events, return updates, seller decisions; GET /api/v1/notifications (unread first)	Backend	1	1.7	Medium
4.2 0	Notification bell UI — badge count, dropdown panel, mark-as-read PATCH endpoint	Frontend	1	4.19	Medium

6.4 Phase 4 Deliverables

- Full cart flow with stock guard, coupon application, and real-time totals
- Multi-step checkout with address, payment (card/UPI/COD), and order confirmation email
- Order lifecycle with StatusHistory, tracking page, and seller order management
- Returns and refunds with photo upload and automated notifications
- Wishlist, coupon engine, and in-app notification system

7. Phase 5 — Advanced Features

Phase 5	AI Chatbot, Reviews & Support System	Weeks 12–14 · ~13 days
---------	--------------------------------------	------------------------

Deliver the AI-powered chatbot, verified-purchase review system, and the complaint/support ticket system. These features complete the customer experience loop and differentiate the platform.

7.1 AI Chatbot Tasks

#	Task	Owner	Est. Days	Dependency	Priority
5.1	Chatbot widget UI (EJS partial) — always-visible fixed bottom-right chat bubble, slide-up panel per UI/UX Brief §3.11	Frontend	1.5	1.6	High
5.2	POST /api/v1/chat/message — receive user message, call AI API (Claude/OpenAI), return assistant reply; persist both messages to ChatHistory	Backend	2	1.7	High
5.3	ChatHistory schema — sessionId (UUID), messages array, TTL 180 days; session resume via GET /api/v1/chat/:sessionId	Backend	1	5.2	High
5.4	Context-aware replies — attach order or product context (context.type, context.refId) when user is on order detail or product page	Backend	1	5.2	Medium
5.5	Chatbot escalation — if AI cannot resolve query, create SupportTicket and link via supportTicketId; set escalated=true on ChatHistory	Backend	1	5.2	Medium

7.2 Reviews & Ratings Tasks

#	Task	Owner	Est. Days	Dependency	Priority
5.6	POST /api/v1/reviews — verified-purchase gate: confirm order ref belongs to customer before creating review; rating 1–5, title, body, images	Backend	1.5	1.7	High
5.7	Review post-save hook — aggregate published reviews for product; update Product.averageRating and Product.reviewCount	Backend	0.5	5.6	High
5.8	GET /api/v1/reviews/product/:productId — paginated published reviews, newest first (compound index)	Backend	0.5	5.6	High
5.9	PUT/DELETE /api/v1/reviews/:id — customer can edit or delete own reviews; re-trigger post-save hook	Backend	0.5	5.6	Medium

#	Task	Owner	Est. Days	Dependency	Priority
5.1 0	Review display on Product Detail Page — star histogram, individual review cards with photos	Frontend	1.5	5.8	High
5.1 1	Unique review enforcement — compound index on (product, customer, order) prevents duplicate reviews	Backend	0.25	1.7	High

7.3 Support Ticket / Complaint Tasks

#	Task	Owner	Est. Days	Dependency	Priority
5.1 2	POST /api/v1/support — create SupportTicket with category, subject, description, attachments (max 3 Cloudinary images), optional order ref	Backend	1	1.7	High
5.1 3	GET /api/v1/support — customer view own tickets; GET /api/v1/support/:id — ticket detail with StatusHistory timeline	Backend	0.5	5.12	High
5.1 4	Complaint UI (EJS) — complaint form with category select, subject, description, photo upload; ticket status tracker per UI/UX Brief §3.12	Frontend	1.5	5.12	High
5.1 5	SupportTicket pre-save middleware — auto-append StatusHistory; set resolvedAt/closedAt on status transitions	Backend	0.5	1.7	High

7.4 Phase 5 Deliverables

- AI chatbot widget with persistent session history and escalation to support
- Verified-purchase review system with auto-updating product rating stats
- Complaint / support ticket system with full status lifecycle and photo evidence

8. Phase 6 — Admin Portal, Polish & Deployment

Phase 6	Admin Portal, Polish & Deployment	Weeks 15–16 · ~10 days
---------	-----------------------------------	------------------------

Build the hidden admin panel covering all governance functions, apply UI polish and accessibility improvements across all screens, implement caching and performance optimisations, and deploy to production. Final QA and security hardening are completed in this phase.

8.1 Admin Portal Tasks

#	Task	Owner	Est. Days	Dependency	Priority
6.1	Hidden admin panel — accessible only via ADMIN_PATH env variable; separate EJS layout with sidebar navigation	Full Stack	1	2.2	High
6.2	Seller approval queue — list pending sellers, approve/reject with email notification; update sellerStatus (BR-02)	Full Stack	1	6.1	High
6.3	User management — list/search users, suspend/ban accounts, view order history	Full Stack	1	6.1	High
6.4	Product moderation — flag/unflag/remove listings; add adminNote; update product status to flagged/pending_review	Full Stack	1	6.1	High
6.5	Complaint management — view/assign/resolve SupportTickets; escalation management	Full Stack	1	5.12	High
6.6	Coupon management — full CRUD for coupons; activate/deactivate toggle	Full Stack	1	4.18	Medium
6.7	Analytics dashboard — platform KPIs: GMV, order count, active users, top products, revenue by period (MongoDB aggregations)	Full Stack	2	6.1	Medium
6.8	Low-stock alert notifications — cron job checks Product.stock < lowStockThreshold; creates low_stock_alert Notification for seller	Backend	0.5	1.7	Medium

8.2 Polish, Performance & Security Tasks

#	Task	Owner	Est. Days	Dependency	Priority
6.9	UI polish pass — Glassmorphism refinements, dark mode testing across all screens, mobile responsiveness audit	Frontend	2	All	High

#	Task	Owner	Est. Days	Dependency	Priority
6.1 0	Server-side caching — node-cache for product listings (60s TTL); cache invalidation on product create/update	Backend	1	3.2	Medium
6.1 1	Security hardening — helmet.js headers, rate limiting (express-rate-limit) on auth routes, input sanitisation (express-validator)	Backend	1	All	High
6.1 2	Tailwind CSS production build — purge unused classes; minify assets; content-hash filenames for cache busting	Frontend	0.5	All	Medium
6.1 3	End-to-end QA — full user journey testing for all 3 roles; edge cases for all 5 business rules	QA	2	All	High
6.1 4	Production deployment — Railway/Render/VPS: PM2 process manager, environment variables, MongoDB Atlas production cluster, Cloudinary production bucket	DevOps	1	All	High
6.1 5	Documentation — API README update, environment variable guide, deployment runbook	Full Stack	0.5	All	Medium

8.3 Phase 6 Deliverables

- Fully functional hidden admin panel with seller approval, user management, moderation, and analytics
- Production-grade security (helmet, rate-limit, input validation)
- Polished UI across all screens with verified dark-mode support
- Platform deployed to production with PM2, Atlas, and Cloudinary

9. API Route Summary

All routes are prefixed `/api/v1`. Admin routes use `{ADMIN_PATH}/api`. Protected routes require a valid `auth_token` JWT cookie.

Authentication — `/api/v1/auth`

Method	Route	Auth	Description
POST	<code>/api/v1/auth/signup</code>	Public	Register new customer account
POST	<code>/api/v1/auth/login</code>	Public	Login, receive JWT cookie
POST	<code>/api/v1/auth/logout</code>	Auth	Clear <code>auth_token</code> cookie
GET	<code>/api/v1/auth/verify/:token</code>	Public	Verify email address
POST	<code>/api/v1/auth/forgot-password</code>	Public	Send password reset email
POST	<code>/api/v1/auth/reset-password/:token</code>	Public	Reset password with token
DELETE	<code>/api/v1/auth/account</code>	Customer	Delete account (BR-03 guard)

Products — `/api/v1/products`

Method	Route	Auth	Description
GET	<code>/api/v1/products</code>	Public	List/search/filter products
GET	<code>/api/v1/products/:slug</code>	Public	Product detail by slug
GET	<code>/api/v1/products/compare</code>	Public	Compare up to 3 products
POST	<code>/api/v1/seller/products</code>	Seller	Create new product listing
PUT	<code>/api/v1/seller/products/:id</code>	Seller	Update own product
DELETE	<code>/api/v1/seller/products/:id</code>	Seller	Unpublish own product

Cart & Wishlist — `/api/v1/cart`

Method	Route	Auth	Description
GET	<code>/api/v1/cart</code>	Customer	Get current cart with totals
POST	<code>/api/v1/cart/add</code>	Customer	Add item (BR-01 stock guard)
PUT	<code>/api/v1/cart/update</code>	Customer	Update item quantity
DELETE	<code>/api/v1/cart/remove/:id</code>	Customer	Remove item from cart
POST	<code>/api/v1/cart/coupon</code>	Customer	Apply coupon code (BR-05)
POST	<code>/api/v1/wishlist/toggle</code>	Customer	Add/remove wishlist item

Method	Route	Auth	Description
GET	/api/v1/wishlist	Customer	Get wishlist

Orders — /api/v1/orders

Method	Route	Auth	Description
POST	/api/v1/orders	Customer	Place order (checkout)
GET	/api/v1/orders	Customer	Order history
GET	/api/v1/orders/:id	Customer	Order detail with tracking
POST	/api/v1/orders/:id/return	Customer	Submit return request
GET	/api/v1/seller/orders	Seller	View orders for seller products
PATCH	/api/v1/seller/orders/:id	Seller	Update order status

Reviews, Chat & Support

Method	Route	Auth	Description
POST	/api/v1/reviews	Customer	Submit verified-purchase review
GET	/api/v1/reviews/product/:id	Public	Get product reviews (paginated)
PUT	/api/v1/reviews/:id	Customer	Edit own review
DELETE	/api/v1/reviews/:id	Customer	Delete own review
POST	/api/v1/chat/message	Customer	Send message to AI chatbot
GET	/api/v1/chat/:sessionId	Customer	Resume chat session
POST	/api/v1/support	Customer	Create support ticket
GET	/api/v1/support	Customer	List own tickets

10. Milestone & Delivery Schedule

The following milestones represent key checkpoints for stakeholder review and sign-off. All estimates assume a team of 3–5 full-stack developers.

Milestone	Phase	Target Week	Deliverable	Status
M1 — Dev Environment Ready	P1	Week 2	All 10 Mongoose models defined; Express server running; MongoDB connected	Planned
M2 — Auth System Live	P2	Week 4	Customer signup/login; seller registration; JWT middleware; role guards active	Planned
M3 — Storefront & Search Live	P3	Week 7	Home page, keyword/voice/image search, product detail, comparison page deployed	Planned
M4 — Seller Portal Complete	P3	Week 7	Seller product CRUD, inventory management, approval workflow end-to-end	Planned
M5 — Commerce Engine Live	P4	Week 11	Cart, checkout, orders, returns/refunds, notifications all functional	Planned
M6 — AI & Reviews Complete	P5	Week 14	Chatbot with session persistence, verified reviews, support ticket system live	Planned
M7 — Admin Panel Complete	P6	Week 15	All admin functions: seller approval, moderation, complaint handling, analytics	Planned
M8 — Production Launch	P6	Week 16	Security hardened, performance optimised, deployed to production with PM2 + Atlas	Planned

10.1 Phase Duration Overview

Phase	Description	Weeks	Duration
Phase 1	Foundation & Setup	1–2	2 weeks
Phase 2	Authentication & Users	3–4	2 weeks
Phase 3	Products & Catalogue	5–7	3 weeks
Phase 4	Commerce Engine	8–11	4 weeks
Phase 5	Advanced Features	12–14	3 weeks
Phase 6	Admin, Polish & Deploy	15–16	2 weeks

11. Risk Register

The following risks have been identified based on the PRD, TRD, and technical complexity assessment. Each risk includes a mitigation strategy to be addressed proactively.

Risk	Likelihood	Impact	Mitigation
AI API rate limits / latency on chatbot	Medium	High	Implement response streaming; add timeout fallback message; queue requests with p-queue; cache common FAQ answers.
Vision API costs for image search exceed budget	Medium	Medium	Implement daily quota cap; use CLIP-based open-source alternative as fallback; rate-limit image search per user.
MongoDB Atlas free-tier index limits	Low	Medium	Audit index count per collection; defer non-critical indexes to Phase 6; upgrade cluster tier if needed.
Payment gateway integration delays (Stripe/Razorpay)	Medium	High	Begin payment integration in Phase 4 Week 1; use sandbox environment; implement COD as guaranteed fallback.
Cloudinary upload failures / quota limits	Low	Medium	Add upload error handling with user-friendly messages; set Cloudinary resource limits; implement local temp storage fallback.
BR-03 account deletion race condition (concurrent orders)	Low	High	Use MongoDB transactions for deletion check + delete; add optimistic locking or status check at time of deletion.
EJS SSR performance under load	Medium	Medium	Cache rendered partials with node-cache; use pagination (max 20 items); add compression middleware.
Seller approval bottleneck (no admin available)	Low	Low	Send admin email on every seller registration; add pending seller count badge to admin dashboard.
JWT token theft via XSS	Low	High	Tokens stored as HttpOnly + Secure + SameSite=Strict cookies; helmet.js CSP headers; DOMPurify on all user-rendered content.
Scope creep on AI features (voice/image search)	High	Medium	Timebox Phase 3 AI tasks to 3 days total; ship keyword search first; mark voice/image as stretch goals with stub fallback.

12. Tech Stack & Dependencies

Package	Version	Purpose
express	5.x	Web framework — routing, middleware, SSR
mongoose	8.x	MongoDB ODM — schemas, queries, middleware, virtuals
ejs	3.x	Server-side HTML templating engine
bcryptjs	2.x	Password hashing (rounds = 12)
jsonwebtoken	9.x	JWT signing and verification (HS256)
cookie-parser	1.x	Parse HttpOnly auth_token cookie on requests
dotenv	16.x	Environment variable loading
multer	1.x	Multipart file upload middleware
cloudinary	2.x	Image/media storage and transformation
nodemailer	6.x	Transactional email — verification, order confirms
tailwindcss	3.x	Utility-first CSS framework with dark mode
helmet	7.x	Security HTTP headers
express-rate-limit	7.x	Rate limiting on auth and API routes
express-validator	7.x	Input validation and sanitisation
morgan	1.x	HTTP request logging
node-cache	5.x	In-memory server-side caching (60s TTL for products)
pm2	5.x	Production Node.js process manager

13. Testing Strategy

13.1 Unit Testing

- Mongoose schema validations — required fields, enum constraints, min/max values
- Service layer business rules — CartService (BR-01), OrderService (BR-04), CouponService (BR-05)
- Middleware — JWT verification, roleGuard, error handler
- Virtuals and instance methods — Coupon.isUsable, Coupon.calculateDiscount(), Cart.subtotal/total

13.2 Integration Testing

- Full auth flow: signup → verify email → login → protected route → logout
- Order placement pipeline: add to cart → apply coupon → checkout → stock decrement → notification created
- Return flow: return_requested → return_accepted → refund_initiated → refunded
- Review flow: order delivered → submit review → product averageRating updated
- Chatbot escalation: unresolved query → SupportTicket created → ChatHistory escalated=true

13.3 Business Rule Verification

Rule	Test Case	Expected Outcome
BR-01	Add out-of-stock product to cart	400 error — "Product out of stock"
BR-02	Unapproved seller creates active product listing	403 Forbidden — seller status check fails
BR-03	Delete account with pending order	400 error — "Cannot delete: pending obligations"
BR-04	Order placement decrements stock to 0	Product.status auto-set to out_of_stock
BR-05	Apply expired coupon at checkout	400 error — coupon.isUsable returns false

14. Definition of Done

A feature is considered "done" only when ALL of the following criteria are satisfied:

Code

- Feature implements the full PRD/TRD specification with no known deviations
- All business rules relevant to the feature are enforced at the correct layer (schema or service)
- Code reviewed by at least one other developer and merged via pull request

Testing

- Unit tests written and passing for service logic and schema middleware
- Integration test covers the happy path and at least one error/edge case
- All 5 business rules pass their verification test cases

UI/UX

- Screen matches the UI/UX Brief specification (layout, color tokens, typography)
- Dark mode renders correctly across all breakpoints (mobile, tablet, desktop)
- Glassmorphism components use correct backdrop-filter and opacity values

Security

- All endpoints protected by appropriate auth + roleGuard middleware
- User-supplied input sanitised via express-validator before processing
- No sensitive data (passwords, tokens) exposed in API responses or logs

Performance

- New MongoDB queries use existing indexes; new indexes added and documented if required
- Cloudinary images served via optimised URL transformations (auto format, quality)
- Page response time under 500ms on local development for standard queries

Documentation

- New API routes documented in the project README with method, path, auth, and request/response shape
- Environment variables documented in .env.example

End of Implementation Plan · E-Commerce Marketplace · v1.0 · June 2026 · Confidential